

Lab 09 — Sorts You Can Watch

DSA27 Lab Manual · Week 9 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 9

Contents

Part 0 — Before you start	2
Part 1 — The pattern: one generator, two uses	4
Part 2 — <code>bubble_sort_steps</code> and <code>bubble_sort</code>	5
Part 3 — <code>selection_sort_steps</code> and <code>selection_sort</code>	6
Part 4 — <code>insertion_sort_steps</code> and <code>insertion_sort</code>	8
Part 5 — <code>counting_sort</code>	9
Part 6 — Count the comparisons	11
Part 7 — The price of watching, and the fix	12
Part 8 — Measure the three sorts	13
Part 9 — Exercises at a glance	13
Part 10 — Take-home practice (not graded)	14
Part 11 — Bridge to Lecture 10: divide, then merge	14
Summary	15
Answers to the checkpoints	16

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means the array after every pass, on paper — and predict at each **Checkpoint** (answers at the end). This week has one extra rule: write each sort **as a generator first**, the `_steps` form, and get the plain sort from it. Then you can watch your own code run, one comparison at a time, before you ask `pytest` whether it is right. Parts 1–5 are the session; Parts 6–8 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
----------	--

You will write

dsa/sorting.py — bubble_sort_steps, selection_sort_steps, insertion_sort_steps, their three plain forms, and counting_sort; and, for yourself, a small Counted class

Graded by

tests/test_sorting.py -k "bubble or selection or insertion or counting or is_sorted" (39 tests)

Connects to

Lecture 09 — Basic sorting; Lecture 08, whose searches need sorted input; and Lecture 10, which sorts the same file in $O(n \log n)$

What you will be able to do

1. draw the array after every pass of bubble, selection and insertion sort, and count the comparisons and the swaps or shifts;
 2. write a sort as a generator that yields a snapshot before each step, and watch it with viz/animate.py;
 3. get the plain sort from the generator without writing the algorithm twice;
 4. say which of the three sorts are stable, and show it with an input;
 5. write counting sort, and say when it is the right sort and when it is not;
 6. count comparisons with a class of your own, and explain the counts with best, worst and average case;
 7. measure what the snapshots cost, and fix it.
-

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with (.venv)):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_sorting.py -q `
    -k "bubble or selection or insertion or counting or is_sorted"
```

The last command shows **38 failed, 1 passed**. The one that passes is test_is_sorted_helper: is_sorted is given. The 38 failures are all NotImplementedError. Merge, quick and heap sort share the file and are Week 10's; the -k filter leaves out their 37 tests, and every command in this lab uses a filter.

0.2 Prerequisites

- **The course Array** (Lecture 02): `Array.from_values(values)` makes a full Array holding a copy of a list; `a[i]` and `a[i] = x` are $O(1)$; `list(a)` turns it back into a list. It has no negative indices and no slicing.
- **Generators** (Lab 03, Part 5): a function with `yield` in it returns a generator; each next runs it to the next `yield` and pauses there.
- **Special methods** (Lab 03): `__lt__` and `__gt__` are what `<` and `>` call. You need them in Part 6.

0.3 Read the skeleton

Open `dsa/sorting.py`. Given: the module docstring (read it — it shows the pattern this lab uses), `__all__`, and `is_sorted`. Yours this week:

Function	Returns	Contract
<code>bubble_sort_steps(values)</code>	a generator of <code>(list, highlight)</code> pairs	yield before each comparison; the last yield is sorted
<code>bubble_sort(values)</code>	a new sorted list	the input list is not changed
<code>selection_sort_steps, selection_sort</code>	the same two forms	
<code>insertion_sort_steps, insertion_sort</code>	the same two forms	stable
<code>counting_sort(values, max_value=None)</code>	a new sorted list	non-negative integers; $O(n + k)$

`highlight` is a tuple of the indices to colour in the animation — the pair about to be compared, for example — or `()` for none.

0.4 The storage rule, this week

A sort is a function, not a structure, but it does need somewhere to work. Copy the input into a course Array — `a = Array.from_values(values)` — sort the Array, and return `list(a)`. That gives you two things at once: the caller's list is never changed (`test_does_not_mutate_the_input`), and the working storage obeys the storage rule. Two more rules, from the point of the week:

- **Do not call a library sort.** No `sorted`, `list.sort`, `min` or `max` of the values inside the three basic sorts. (`counting_sort` may use `max` to find `k` — the lecture's step 1.)
- **Write each algorithm once.** The plain form must reuse the `_steps` form, not copy its loops (Part 1).

Part 1 — The pattern: one generator, two uses

Lecture 09, “The code, written as a generator” and “One yield, one frame”. The lecture shows `bubble_sort_steps` in full. Read it, close it, and write it from memory in Part 2; here you learn to **use** such a generator before you write one.

1.1 Watch one

Open `notebooks/09-sorting-basic.ipynb`. Its “Try it” cell holds a small `bubble_sort_steps` of its own (on a Python list, with no early exit). Run it: the slider steps through the sort one comparison at a time. Then try the other view:

```
from IPython.display import HTML
from viz.animate import animate_bars
HTML(animate_bars(bubble_sort_steps([5, 2, 9, 1, 7])).to_jshtml())
```

1.2 One generator, three consumers

A generator does nothing until something iterates it. Try, in the notebook:

```
frames = bubble_sort_steps([5, 2, 9, 1, 7])
print(frames)                    # a generator object: nothing has run yet
first = next(frames)             # runs to the first yield, and pauses
print(first)
print(sum(1 for _ in frames))    # runs the rest, counting what is left
```

That is the whole trick. The **animation** iterates the generator and draws each frame; a **count** iterates it and adds 1 per frame; and the **plain sort** iterates it and keeps only the last frame. So the plain form is three lines, and the module docstring of `dsa/sorting.py` shows them. Write a small helper for it once, and use it in all three plain sorts:

```
def _finish(steps):
    """Run a _steps generator to the end and return its last state."""
```

It must work when the generator yields only once or twice (the empty list), and it must return a **list**.

Checkpoint 1. For the lecture’s `bubble_sort_steps` — one yield before the loops, one before each comparison, one after — how many frames does each call produce?

- (a) `bubble_sort_steps([5, 2, 9, 1, 7])`
- (b) `bubble_sort_steps([1, 2, 3, 4, 5])`
- (c) `bubble_sort_steps([])`

And (d): the notebook’s version yields the **same list object** every time. Why does `step_slider` still show different frames, while `frames = list(bubble_sort_steps([5, 2, 9, 1, 7]))`, with the notebook’s version, shows the sorted list in every frame?

Part 2 — bubble_sort_steps and bubble_sort

Lecture 09, “Bubble sort”. Compare neighbours; swap them when the left one is larger. Each pass carries the largest remaining value to the end, so each pass can stop one slot earlier. A pass with no swap means the array is sorted: stop.

2.1 Draw it

On paper, one row per pass: the array after the pass, and how many comparisons and swaps it made. Shade the slots that are in their final place.

You can check each row in the notebook:

```
from viz.draw import draw_array
draw_array([1, 3, 4, 7, 2, 9], done=[5], title="after pass 1")
```

Checkpoint 2. Trace bubble sort, with the early exit, on [4, 1, 3, 9, 7, 2].

- Write the array after each pass, with its comparisons and swaps.
- How many passes run, and why does the last one not stop earlier?
- Total comparisons and total swaps. Count the **inversions** of the input (pairs in the wrong order) and compare.

2.2 Write it

- Copy the input into an Array, and remember n .
- Yield the first state: `list(a), ()`.
- For each pass: the pass looks at `a[0..end]`, with `end` going from $n - 1$ down to 1. Set `swapped = False` **at the start of each pass**.
- For each j from 0 while $j + 1 \leq \text{end}$: **yield** `list(a), (j, j + 1)`, then compare and, if needed, swap and set `swapped`.
- After the pass, if nothing was swapped, stop.
- Yield the last state.

Then `bubble_sort` is one line, with your `_finish`.

Hints:

- Yield `list(a)`, never `a`. The animation keeps every frame; a frame must be a copy of the state **now**, not the Array that will change after the yield.
- `a[j], a[j + 1] = a[j + 1], a[j]` swaps in one line, and it works on the course Array as it does on a list.

2.3 Test it

```
pytest tests/test_sorting.py -v -k bubble
```

Twelve tests: the eight fixed cases of `test_sorts_correctly` (empty, one, two, [5, 2, 9, 1, 7], three equal values, sorted, reversed, negatives), `test_does_not_mutate_the_input`,

test_sorts_random_lists (20 random lists, fixed seed), and the two tests on the _steps form.

2.4 When it fails

Bug	What you see	Fix
range(end + 1) in the inner loop	IndexError: Array index 5 out of range for length 5 (valid: 0..4; no negative indices) — 10 of the 12 fail	the pass compares a[j] with a[j + 1], so j stops at end - 1
the if not swapped: break indented inside the inner loop	assert [2, 5, 1, 7, 9] == [1, 2, 5, 7, 9] — 4 fail	the check belongs after the pass, not after one comparison
a = values instead of a copy	only test_does_not_mutate_the_input fails: sort a copy — callers do not expect their list changed	Array.from_values(values)
the last yield is a, () instead of list(a), ()	assert [Array([1, 2, 5, 7, 9]), ()] == [1, 2, 5, 7, 9] in test_step_form_is_a_generator_ending_sorted	yield list(a)
bubble_sort returns bubble_sort_steps(values)	assert <generator object ...> == [3, 3, 3] — 9 fail	run the generator to the end: _finish
swapped = False before the outer loop	nothing fails	see below

The last bug is the dangerous one: all twelve tests pass. Once any pass has swapped, swapped stays True for ever, so the early exit never fires again and bubble sort always does all $n - 1$ passes. The output is right; the best case is gone. Only a count finds it — Part 6 will.

Part 3 — selection_sort_steps and selection_sort

Lecture 09, “Selection sort”. Round i finds the smallest value in $a[i..n-1]$ and swaps it into slot i .

3.1 Draw it

One row per round: the array after the round, the index of the minimum, and whether a swap happened.

Checkpoint 3. (a) Trace selection sort on [4, 1, 3, 9, 7, 2]: the array after each round, the comparisons per round, and the swaps. Compare the totals with Checkpoint 2.

(b) Sort the three cards 2a, 2b, 1c — key first, label second, compared by key only — with selection sort. Is the result stable? Which line of the algorithm is to blame?

3.2 Write it

1. Copy, yield the first state.
2. For each i from 0 to $n - 2$: `smallest = i`; for each j after i , **yield** `list(a), (smallest, j)` and then compare `a[j]` with `a[smallest]` — the best so far, not `a[i]`.
3. After the scan, swap `a[i]` and `a[smallest]` **only if** they differ, and yield the state after the swap.
4. Yield the last state. `selection_sort` is one line.

3.3 Test it

```
pytest tests/test_sorting.py -v -k selection
```

Twelve tests, the same set as bubble sort's.

3.4 When it fails

Bug	What you see	Fix
<code>if a[j] < a[i]:</code>	<code>assert [1, 2, 7, 5, 9] == [1, 2, 5, 7, 9] — 4 fail</code>	compare with the smallest so far : <code>a[smallest]</code>
the swap inside the inner loop	nothing fails, but it swaps every time it finds a smaller value	one swap per round, after the scan: that is the point of selection sort
<code>for j in range(i, n)</code>	nothing fails	it compares <code>a[i]</code> with itself once per round: harmless, and wasted

The swap-inside-the-loop version is a different, worse algorithm that the tests cannot tell apart: it sorts, but it writes up to $n^2/2$ times instead of at most $n - 1$. Count its swaps once you have Part 6.

Part 4 — insertion_sort_steps and insertion_sort

Lecture 09, “Insertion sort” and “Insertion sort counts inversions”. Grow a sorted prefix: take `current = a[i]`, shift every larger value in `a[0..i-1]` one slot right, and drop `current` into the hole.

4.1 Draw it

One row per round: the array after inserting `a[i]`, and how many shifts it took.

Checkpoint 4. (a) Trace insertion sort on `[4, 1, 3, 9, 7, 2]`: the array after each round, the shifts, and the comparisons per round. Compare the total shifts with the inversions you counted in Checkpoint 2.

(b) How many comparisons does insertion sort make on `[1, 2, 3, 4, 5, 6]`? On `[6, 5, 4, 3, 2, 1]`?

(c) The loop condition is `j >= 0` and `a[j] > current`. A student writes `a[j] > current` and `j >= 0`. On a Python **list**, what would the test `a[j] > current` read when `j` is `-1`? What happens on the course Array?

4.2 Write it

1. Copy, yield the first state.
2. For each `i` from 1 to `n - 1`: `current = a[i]`, `j = i - 1`.
3. While `j` is a valid index **and** `a[j] > current`: move `a[j]` one slot right, move `j` left.
4. Put `current` at `j + 1`.
5. Yield after each shift, with a highlight of your choice, and yield the last state. `insertion_sort` is one line.

A choice to make: after a shift, the array holds the shifted value **twice** until `current` is dropped in. A frame taken then shows a duplicate. The reference writes `current` into the hole after every shift, so every frame is a permutation of the input; that costs one extra write per shift. Either is accepted — say which you chose, and why, when the TA asks.

4.3 Test it

```
pytest tests/test_sorting.py -v -k insertion
```

Thirteen tests: the twelve of the other two sorts, and `test_insertion_sort_is_stable`. That test sorts four Card objects that compare **by key only**, so that two cards with the same key are equal to the sort and still distinguishable by their label.

4.4 When it fails

Bug	What you see	Fix
<code>a[j] > current</code> and <code>j >= 0</code>	<code>IndexError: Array index -1 out of range for length 5 (valid: 0..4; no negative indices)</code> — 9 of 13 fail	test <code>j >= 0</code> first : and stops before the read
<code>a[j] >= current</code>	only <code>test_insertion_sort_is_stable</code> fails: <code>assert ['0d', '0b', '1c', '1a'] == ['0b', '0d', '1a', '1c']</code>	strictly greater: an equal key trails stop the walk
no <code>a[j + 1] = current</code> after the loop	<code>assert [5, 5, 5, 9, 9] == [1, 2, 5, 7, 9]</code> — values duplicated, others lost; 7 fail	the shifted-over value is still there; current must be written back

The first bug is worth a moment. On a Python list, `a[-1]` is the **last** element, so the reversed condition reads a value that has nothing to do with the sort; then `j >= 0` is false and the loop stops anyway. The answer comes out right — by luck — and the bug stays in the code, waiting for a language where `a[-1]` is a crash or garbage (C). The course `Array` refuses negative indices, so you see the bug on the exact line, the first time it happens.

Part 5 — counting_sort

Lecture 09, “Counting sort”. Values are non-negative integers. Count how many times each value occurs; then write each value out, in increasing order, as many times as it was counted. No two values are ever compared.

5.1 Draw it

Draw three rows: the values, the counts array with its indices $0 \dots k-1$ above it, and the output.

Checkpoint 5. (a) For `counting_sort([3, 0, 2, 3, 0, 3])`: what is k , what is counts, and what is the output?

(b) `counting_sort([2, 1_000_000])` sorts two numbers. How many counters does it make, and how much work does the write-out loop do? What does that say about when counting sort is a good idea?

(c) Why can counting sort not sort `[2.5, 1.0]` or `['b', 'a']` as written?

5.2 Write it

1. The empty list: return `[]` at once — `max([])` raises `ValueError`.
2. Check every value is a non-negative int; otherwise raise `ValueError`.
3. `k` is `max_value + 1` if `max_value` was given, else `max(values) + 1`.
4. `counts = Array(k, fill=0)`; one pass: `counts[v] += 1`.
5. An output `Array` of `len(values)` slots; for each `v` from 0 to `k - 1`, write `v` into the next free slot `counts[v]` times. Return it as a list.

5.3 Test it

```
pytest tests/test_sorting.py -v -k counting
```

One test, `test_counting_sort`, with two cases: `[3, 1, 4, 1, 5, 0]` and `[]`. That is thin, so test it yourself against sorted on random lists of small integers, as Part 8 of Lab 08 did with `bisect`:

```
import random
from dsa.sorting import counting_sort
rng = random.Random(1)
for trial in range(1000):
    values = [rng.randint(0, 20) for _ in range(rng.randint(0, 30))]
    assert counting_sort(values) == sorted(values), values
print('1,000 random lists: all agree')
```

5.4 When it fails

Bug	What you see	Fix
<code>k = max(values), without + 1</code>	<code>IndexError: Array index 5 out of range for length 5</code>	values run from 0 to max inclusive : <code>max + 1</code> counters
no empty-list check	<code>ValueError: max() iterable argument is empty</code>	return <code>[]</code> before calling <code>max</code>

5.5 All 39 at once

```
pytest tests/test_sorting.py -v `
-k "bubble or selection or insertion or counting or is_sorted"
```

Part 6 — Count the comparisons

Lecture 09, “Counting comparisons and writes”. A timing depends on your machine; a **count** does not.

6.1 Write Counted

Not graded, and not in dsa/: write it at the end of the notebook. The contract:

- `Counted(v)` keeps a value `v`;
- `a < b` and `a > b` compare the two `vs`, and add 1 to a counter **shared by all** `Counted` objects — a class attribute, `Counted.compares`.

Your sorts only ever use `<` and `>` on the values, so they cannot tell a list of `Counted` from a list of numbers.

Checkpoint 6. Predict `Counted.compares` for $n = 100$, for each of your three sorts, on sorted input and on reversed input (six numbers).

6.2 All three sorts, four inputs

```
import random
from dsa import sorting

def comparisons(sort, values):
    Counted.compares = 0
    result = sort([Counted(v) for v in values])
    assert [c.v for c in result] == sorted(values)
    return Counted.compares

n = 1000
rng = random.Random(9)
nearly = list(range(n))
for _ in range(10):                                # swap 10 random neighbours
    i = rng.randrange(n - 1)
    nearly[i], nearly[i + 1] = nearly[i + 1], nearly[i]
inputs = {'sorted': list(range(n)), 'nearly': nearly,
          'random': rng.sample(range(n), n),
          'reversed': list(range(n, 0, -1))}
for sort in (sorting.bubble_sort, sorting.selection_sort,
             sorting.insertion_sort):
    counts = {name: comparisons(sort, v) for name, v in inputs.items()}
    print(f'{sort.__name__:15}', counts)
```

With the reference solution this prints:

	sorted	nearly	random	reversed
bubble_sort	999	1,997	491,874	499,500
selection_sort	499,500	499,500	499,500	499,500
insertion_sort	999	1,009	249,614	499,500

Yours should match exactly — the three algorithms are fully determined. If your `bubble_sort` shows 499,500 on sorted input, its early exit is broken: the swapped bug of Part 2.4. If your `insertion_sort` shows more than 999 on sorted input, its walk does not stop at the first value that is not greater.

Part 7 — The price of watching, and the fix

Lecture 09, “The price of watching”. Your plain sorts run the `_steps` generator to the end, and each `yield` makes a `list(a)` copy — $O(n)$ — before each of about $n^2/2$ comparisons.

7.1 Measure it

```
import random
from viz.complexity import measure, plot_growth
from dsa import sorting

sizes = [50, 100, 200, 400]
make = lambda n: random.Random(n).sample(range(n), n)
steps = measure(sorting.bubble_sort, sizes, make, repeat=1, warmup=False)
plot_growth({'bubble_sort': steps}, reference=['n^2', 'n^3'], loglog=True)
print([round(b / a, 1) for a, b in zip(steps[1], steps[1][1:])])
```

Checkpoint 7. Before you run it: if the plain sort is $O(n^2)$, by what factor should the time grow each time n doubles? And if it is $O(n^3)$? Which do you see, and why?

7.2 Fix it

Give each `_steps` function a parameter `snapshot=list`, and `yield snapshot(a)` instead of `list(a)`. The animation calls it with the default and gets copies, as before. The plain sort passes a function that returns the `Array` itself, with no copy at all; your `_finish` already turns the last state into a list. Run the tests again (all 39 must pass), then the measurement: the ratios drop from about 8 to about 4.

Part 8 — Measure the three sorts

```
sizes = [250, 500, 1000, 2000]
make = lambda n: random.Random(n).sample(range(n), n)
results = {sort.__name__: measure(sort, sizes, make, repeat=1, warmup=False)
           for sort in (sorting.bubble_sort, sorting.selection_sort,
                       sorting.insertion_sort, sorting.counting_sort)}
plot_growth(results, reference=['n^2', 'n'], loglog=True)
```

It takes about half a minute. What to expect (Lecture 09, “Measured time”):

- the three $O(n^2)$ sorts are parallel lines of slope 2 on the log-log plot: double n , four times the time;
- selection sort is usually the fastest of the three **in Python**, although it compares the most — it writes almost nothing, and a swap costs more than a comparison here;
- counting sort is a line of slope 1, far below the others.

Then time `insertion_sort` alone on sorted, nearly sorted, random and reversed input of the same size (the four inputs of Part 6, at $n = 2000$). Nearly sorted is hundreds of times faster than random. That is why real library sorts finish small or nearly sorted pieces with insertion sort (Lecture 09, “Insertion sort on nearly sorted input”).

Part 9 — Exercises at a glance

Function	Target cost	The trap	-k filter
<code>bubble_sort_steps</code>	$O(n^2)$, $O(n)$ best	<code>range(end)</code> ; reset swapped each pass; <code>yield list(a)</code>	<code>bubble</code> (12)
<code>selection_sort_steps</code>	$O(n^2)$ always, at most $n - 1$ swaps	compare with <code>a[smallest]</code> ; swap after the scan	<code>selection</code> (12)
<code>insertion_sort_steps</code>	$O(n^2)$, $O(n)$ best	<code>j >= 0</code> first; <code>></code> not <code>>=</code> ; write current back	<code>insertion</code> (13)
the three plain sorts	as above	reuse the generator; no copy per step (Part 7)	(in the three above)
<code>counting_sort</code>	$O(n + k)$	<code>k = max + 1</code> ; the empty list	<code>counting</code> (1)

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in the three basic sorts calls `sorted`, `sort`, `min` or `max`, and that the plain sorts do not repeat the loops of the `_steps` functions.

Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week09-questions.md`, with answers in `week09-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week09.py`: `count_inversions`, `sort_by_key`, `dutch_flag`, `sort_k_sorted` and `counting_sort_by_key` (W9-C1 to W9-C5). The tests count comparisons or reads for the last three, so an $O(n^2)$ answer to an $O(n)$ or $O(nk)$ problem fails:

```
pytest tests/test_practice_week09.py -v
```

Hint for `counting_sort_by_key`: turn the counts into **starting positions** with a running sum, then place the items in input order.

2. **W9-S1** — draw every pass of bubble sort on `[6, 5, 3, 1, 8, 7, 2, 4]`.
3. **W9-T4** — how many frames does each `_steps` generator yield on `[5, 4, 3, 2, 1]`? Predict, then check with `len(list(...))`.
4. **W9-B2** — a selection sort that swaps inside the inner loop. Count its swaps on reversed input of size 6, and compare with the real one.
5. **W9-E5** — the comparison lower bound in your own words, and why counting sort does not contradict it.

The worked solutions are in `solutions/dsa/sorting.py` and `solutions/practice/week09.py` — for after you have tried. `pytest --solutions tests/test_practice_week09.py` runs the tests on them.

Part 11 — Bridge to Lecture 10: divide, then merge

Insertion sort costs about $n^2/4$ comparisons on random input. What if you sorted two **halves** separately, and then combined them? Each half costs about $(n/2)^2/4 = n^2/16$, so both together cost $n^2/8$ — half as much. Combining two sorted lists is the `merge_sorted` you wrote in Week 2 (`dsa/array_ops.py`): one pass, at most $n - 1$ comparisons.

```
import random
from dsa import sorting
from dsa.array_ops import merge_sorted

values = random.Random(1).sample(range(1000), 1000)

Counted.compares = 0
sorting.insertion_sort([Counted(v) for v in values])
print('whole', Counted.compares)

Counted.compares = 0
```

```

left = sorting.insertion_sort([Counted(v) for v in values[:500]])
right = sorting.insertion_sort([Counted(v) for v in values[500:]])
merged = merge_sorted(left, right)
print('two halves, then merge', Counted.compares)

```

(merge_sorted compares with $\leq$, so give Counted a `__le__` too, counting like the others.) With the reference solutions: **251,387** comparisons for the whole list, **125,992** for two halves and a merge. Split into four quarters and merge twice, and it drops to 65,733. Two questions to bring to Lecture 10:

- **Why stop at four?** Split the halves in half, and those in half again, until the pieces have one element — which is sorted already. What is left is only merging. That is **merge sort**, and it costs $O(n \log n)$.
- **Is it stable?** merge_sorted takes from the left list when the two front values are equal. Why does that one choice make the whole sort stable?

Summary

Idea	The one line to keep
<code>_steps</code> generator	Yield a snapshot before each step; the animation, a count and the plain sort all iterate it.
Plain form	Run the generator to the end and keep the last state — one algorithm, not two.
Snapshots	<code>list(a)</code> per frame; but not in the plain sort, or $O(n^2)$ becomes $O(n^3)$.
Bubble sort	Neighbours swap; reset swapped each pass; best case $O(n)$.
Selection sort	$n(n-1)/2$ comparisons always; at most $n-1$ swaps; not stable.
Insertion sort	One shift per inversion; <code>j >= 0</code> first; <code>></code> keeps it stable.
Stability	Only visible with records that compare equal but are different.
Counting sort	$O(n+k)$, no comparisons; $k = \max + 1$; bad when k is huge.
Measuring	Count comparisons with a class whose <code><</code> and <code>></code> count.

Answers to the checkpoints

Checkpoint 1. (a) **12**: one at the start, ten comparisons ($4 + 3 + 2 + 1$ — the fourth pass swaps nothing), one at the end. (b) **6**: the first pass makes 4 comparisons, swaps nothing, and stops. (c) **2**: the start and the end, both `[]`; the loops do not run. (d) `step_slider` (through `normalise_frames`) copies each frame **as it arrives**, while the list still holds that step's values. `list(generator)` only collects references: eleven frames (the notebook's version has no first yield and no early exit: $4 + 3 + 2 + 1$ comparisons and one final yield), every one holding the **same** list, which by the end is `[1, 2, 5, 7, 9]`. That is why your own generator must yield `list(a)`: a frame must not depend on who is iterating, or when.

Checkpoint 2.

start	4	1	3	9	7	2	
after pass 1	1	3	4	7	2	9	5 comparisons, 4 swaps
after pass 2	1	3	4	2	7	9	4 comparisons, 1 swap
after pass 3	1	3	2	4	7	9	3 comparisons, 1 swap
after pass 4	1	2	3	4	7	9	2 comparisons, 1 swap
after pass 5	1	2	3	4	7	9	1 comparison, 0 swaps

(b) All five passes run. The early exit needs a pass with **no** swap, and pass 4 still swapped (3 with 2); pass 5 is the last possible pass anyway.

(c) **15 comparisons, 7 swaps**. The inversions are (4,1), (4,3), (4,2), (3,2), (9,7), (9,2) and (7,2): seven. Every swap of neighbours fixes exactly one inversion, so bubble sort always makes exactly as many swaps as there are inversions.

Checkpoint 3. (a)

start	4	1	3	9	7	2	
i = 0	1	4	3	9	7	2	5 comparisons; min at 1: swap
i = 1	1	2	3	9	7	4	4 comparisons; min at 5: swap
i = 2	1	2	3	9	7	4	3 comparisons; min at 2: no swap
i = 3	1	2	3	4	7	9	2 comparisons; min at 5: swap
i = 4	1	2	3	4	7	9	1 comparison; min at 4: no swap

15 comparisons and 3 swaps, against 15 and 7 for bubble sort: the same comparisons (here), far fewer writes. (b) 1c, 2b, 2a — **not stable**. Round 0 finds 1c at index 2 and swaps it with 2a at index 0, which sends 2a to the back, behind 2b. The culprit is the long-distance **swap** in step 3. (Bubble and insertion sort both give 1c, 2a, 2b.)

Checkpoint 4. (a)

start	4	1	3	9	7	2	
i = 1	1	4	3	9	7	2	1 shift, 1 comparison
i = 2	1	3	4	9	7	2	1 shift, 2 comparisons
i = 3	1	3	4	9	7	2	0 shifts, 1 comparison
i = 4	1	3	4	7	9	2	1 shift, 2 comparisons
i = 5	1	2	3	4	7	9	4 shifts, 5 comparisons

7 shifts — exactly the 7 inversions of Checkpoint 2 — and 11 comparisons. Each round compares once more than it shifts, unless the walk runs off the front (round 1, and round 5 would have if current were smaller than 1). (b) Sorted: **5** comparisons, one per round, no shifts. Reversed: **15**, every pair — $n(n-1)/2$. (c) On a list, `a[-1]` is the **last** element: the loop compares current with a value from the wrong end of the list. Then `j >= 0` is false, so the loop stops anyway and the result is still right — the bug is invisible on a list. The course Array raises `IndexError: Array index -1 out of range ... no negative indices on that line`.

Checkpoint 5. (a) `k = 4`; `counts = [2, 0, 1, 3]` (two 0s, no 1, one 2, three 3s); output `[0, 0, 2, 3, 3, 3]`. (b) **1,000,001** counters, and the write-out loop visits every one of them to write two values. $O(n+k)$ with $n=2$ and k a million: counting sort is a good idea only when k , the range of the values, is not much larger than n . (c) The value is used as an **index**: `counts[2.5]` has no meaning, and nor does `counts['b']`. The reference raises `ValueError`. Letters can be sorted by counting after mapping them to 0..25 — practice problem W9-C5.

Checkpoint 6.

	sorted	reversed
bubble (early exit)	99	4,950
selection	4,950	4,950
insertion	99	4,950

Sorted input: bubble sort's first pass swaps nothing and stops after $n-1$; insertion sort makes one comparison per round. Reversed: every pair is compared, $100 \times 99/2 = 4950$. Selection sort does not look at the data to decide how much to do: 4,950 always.

Checkpoint 7. $O(n^2)$: $\times 4$ per doubling. $O(n^3)$: $\times 8$. With a copy per frame you see about $\times 8$ (the reference measured 6.7, 7.1 and 8.1 on these sizes): each of the $n(n-1)/2$ yields copies n values. After the fix, about $\times 4$.